Cerebro Atlas Methodology

Public docs URL: https://cerebro-finnie.vercel.app/docs

This guide explains how to recreate Cerebro Atlas from scratch without GitHub access. It packages the operating model, the build steps, and the key source files in one Markdown document so it can be emailed directly or exported to PDF.

The problem

Most LLM + document stacks still behave like one-shot retrieval systems. Every question triggers a fresh search over raw files, the model synthesizes an answer, and then the work disappears. Andrej Karpathy's "LLM Wiki" framing argues for a different pattern: the model should maintain a persistent intermediate knowledge layer instead of rediscovering the same ideas every time. That idea is a modern answer to Vannevar Bush's Memex problem: how do you keep institutional memory linked, updated, and queryable over time?

Cerebro Atlas implements that pattern as a local-first Obsidian vault plus a derived graph snapshot. The vault compounds. The graph makes the topology legible. The LLM's job is not only to answer questions; it is to keep the map current.

The three layers

1. Raw sources. These are immutable inputs such as meeting notes, research artifacts, transcripts, screenshots, internal documents, and personal reflections.
2. The wiki. This is the LLM-maintained Markdown layer stored in the Obsidian vault. It contains entity pages, summaries, topic pages, references, and cross-links.
3. The schema. This is the lightweight operating contract that tells the model how to ingest, organize, and query the vault. In practice this is a `CLAUDE.md` -style file plus build conventions.

Obsidian vault setup

The active vault used for this deployment is `C:/Users/james/Desktop/CerebroAtlas` . The builder currently sees 555 Markdown files, including 285 inside `pm-brain-main` . CerebroFinnie itself lives in a separate app repo and points to the vault through `cerebro.config.json` .

A practical folder layout looks like this:

```
vault-root/
|-- 00-Identity/
|-- 01-Projects/
|-- 02-Prompts/
|-- 03-Snapshots/
|-- 04-LLM-Notes/
|-- 05-Conversations/
|-- 06-Index/
|-- 07-Life/
|-- 08-Work/
|-- 09-People/
|-- 10-Timeline/
`-- raw/
```

Key conventions:

- Use plain Markdown files with YAML frontmatter.
- Use Obsidian wikilinks like `[[Note Title]]` and aliases like `[[Full Title|Short Label]]`.
- Keep tags in either frontmatter or inline `#tag/path` form.
- Use Obsidian graph color groups when you want manual overrides on top of topology-based clustering.
- Treat `note.folder` as metadata only. The topology now comes from graph density, not from folder names.

Example frontmatter:

```
---
title: CIBC - Product Strategy 2026
tags: [work/cibc, strategy, q1]
updated: 2026-03-15
aliases: [CIBC Strategy, cibc-strategy]
---
```

Minimal repo config:

```
{
  "vaultPath": "../../CerebroAtlas",
  "excludeDirectories": ["archive", "raw"],
  "excludeFiles": ["CLAUDE.md", "README.md"]
}
```

The build script

`scripts/build-vault-graph.mjs` is the indexing boundary between the vault and the viewer. It performs five jobs:

1. Walk the vault and collect Markdown files.
2. Parse frontmatter and content, with SHA256 caching to skip unchanged files.
3. Resolve explicit wikilinks into graph edges.
4. Merge optional semantic edges from graphify output.
5. Run Louvain on the combined topology and emit `public/data/vault-graph.json`.

Importance scoring is deliberately simple and legible:

```
importance =
  incomingLinks * 1.7
+ outgoingLinks * 1.15
+ tagCount * 0.45
+ min(wordCount / 280, 2.5)
```

For orphan-heavy folders, the builder also synthesizes low-weight sibling edges so isolated note runs still cluster visually.

Full `scripts/build-vault-graph.mjs`

```
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import matter from 'gray-matter';
import Graph from 'graphology';
import louvain from 'graphology-communities-louvain';
import { importGraphifyEdges } from './import-graphify-edges.mjs';
import { sha256, getCachedByHash, saveCacheByHash, evictStaleCache } from './cache-
manager.mjs';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const APP_ROOT = path.resolve(__dirname, '..');
const OUTPUT_PATH = path.join(APP_ROOT, 'public', 'data', 'vault-graph.json');
const CONFIG_PATH = path.join(APP_ROOT, 'cerebro.config.json');
const DEFAULT_EXCLUDED_DIRECTORIES = [
  '.git',
  '.obsidian',
  '.claude',
  '.codex',
  '.next',
  '.turbo',
  '.vercel',
  'node_modules',
  'coverage',
  'build',
  'dist',
  'out',
  'viewer',
  path.basename(APP_ROOT),
];

// Expanded palette - more distinct colors for topology-detected communities.
// Louvain may produce 20-50 communities across a large vault; cycle through these.
const DEFAULT_COLORS = [
  '#4c7f76', '#ce7f38', '#cd5c4e', '#4b69b1', '#8b6eb8',
  '#6d8f4e', '#cf9e36', '#6a7f98', '#9c5e80', '#3d8f92',
  '#e57373', '#81c784', '#64b5f6', '#ffb74d', '#f06292',
  '#4db6ac', '#7986cb', '#a1887f', '#ffd54f', '#90a4ae',
  '#b39ddb', '#80cbc4', '#ef9a9a', '#a5d6a7', '#90caf9',
];

const WIKILINK_PATTERN = /\[\[([^\]|#]+)(?:#[^\]|]+)?(?:\|[^\]]+)?\]\]/g;
const INLINE_TAG_PATTERN = /(^|\s)#([A-Za-z0-9_/-]+)/g;

async function main() {
  const config = await readConfig();
  const vaultRoot = await resolveVaultRoot(config);
  const excludedDirectories = buildExcludedDirectories(config);
  const excludedFiles = buildExcludedFiles(config);
  const obsidianGraphPath = path.join(vaultRoot, '.obsidian', 'graph.json');
```

```
  const files = await collectMarkdownFiles(vaultRoot, vaultRoot, excludedDirectories,
excludedFiles);
  const graphSettings = await readGraphSettings(obsidianGraphPath);
  const notes = [];
  const activeHashes = new Set();
  let cacheHits = 0;

  for (const file of files) {
    const raw = await fs.readFile(file, 'utf8');
    const relativeFile = path.relative(vaultRoot, file);
    const notePath = normalizePath(relativeFile.replace(/\.md$/i, ''));

    // Compute hash once; track it so eviction knows this file is still live.
    const hash = sha256(raw);
    activeHashes.add(hash);

    // Try cache first - only re-parse if the file content changed.
    const cached = await getCachedByHash(hash);
    if (cached) {
      // Patch the path fields in case the file was renamed (content unchanged).
      cached.id = notePath;
      cached.path = notePath;
      cached.folder = notePath.includes('/') ? notePath.split('/')[0] : 'Vault Root';
      cached.sourceFile = normalizePath(relativeFile);
      notes.push(cached);
      cacheHits += 1;
      continue;
    }

    const parsed = matter(raw);
    const title = extractTitle(parsed.content, path.basename(notePath));
    const aliases = toStringList(parsed.data.aliases);
    const frontmatterTags = toStringList(parsed.data.tags);
    const inlineTags = extractInlineTags(parsed.content);
    const tags = Array.from(new Set([...frontmatterTags, ...inlineTags])).sort();

    const note = {
      id: notePath,
      title,
      path: notePath,
      folder: notePath.includes('/') ? notePath.split('/')[0] : 'Vault Root',
      aliases,
      tags,
      rawLinks: extractLinks(parsed.content),
      excerpt: extractExcerpt(parsed.content),
      markdown: parsed.content.trim(),
      fullMarkdown: raw.trim(),
      wordCount: countWords(parsed.content),
      updated: parsed.data.updated ?? null,
      sourceFile: normalizePath(relativeFile),
    };
```

```
    await saveCacheByHash(hash, note);
    notes.push(note);
  }

  // Prune cache entries for files that are no longer in the vault.
  await evictStaleCache(activeHashes);

  if (cacheHits > 0) {
    console.log(`  Cache: ${cacheHits}/${files.length} files served from cache
(unchanged)`);
  }

  const notesById = new Map(notes.map((note) => [note.id, note]));
  const aliasMap = buildLookupMap(notes, (note) => note.aliases);
  const basenameMap = buildLookupMap(notes, (note) => [path.basename(note.path)]);
  const resolvedEdges = [];
  const seenEdges = new Set();

  //  Wikilink edges
  for (const note of notes) {
    const outgoing = [];

    for (const rawTarget of note.rawLinks) {
      const resolved = resolveNoteTarget(rawTarget, note.path, notesById, basenameMap,
aliasMap);
      if (!resolved || resolved === note.id) {
        continue;
      }

      outgoing.push(resolved);

      const edgeKey = `${note.id}=>${resolved}`;
      if (!seenEdges.has(edgeKey)) {
        seenEdges.add(edgeKey);
        resolvedEdges.push({ source: note.id, target: resolved, weight: 1, kind: 'wikilink'
});
      }
    }

    note.outgoing = Array.from(new Set(outgoing)).sort();
  }

  //  Incoming counts (wikilinks only at this stage)
  const incomingCounts = new Map(notes.map((note) => [note.id, 0]));
  for (const edge of resolvedEdges) {
    incomingCounts.set(edge.target, (incomingCounts.get(edge.target) ?? 0) + 1);
  }

  //  Semantic edges from graphify (optional)
  // If the vault contains a graphify-out/graph.json, merge those edges in.
  // They carry real LLM-extracted relationships between notes, tagged as
  // kind: 'semantic' with a confidence field (EXTRACTED / INFERRED / AMBIGUOUS).
```

```
  const semanticEdges = await importGraphifyEdges(vaultRoot, notes, seenEdges);
  for (const edge of semanticEdges) {
    resolvedEdges.push(edge);
    incomingCounts.set(edge.target, (incomingCounts.get(edge.target) ?? 0) + 1);
    const sourceNote = notesById.get(edge.source);
    if (sourceNote) {
      sourceNote.outgoing = [...new Set([...(sourceNote.outgoing ?? []),
edge.target])].sort();
    }
  }
  if (semanticEdges.length > 0) {
    console.log(`  Merged ${semanticEdges.length} semantic edges from graphify`);
  }

  //  Sibling edges for orphan-heavy folders
  // For folders where >=60% of notes have no links at all, synthesize lightweight
  // sibling edges so the force layout doesn't scatter them randomly. Louvain
  // uses these edges to detect the community structure.
  const folderBuckets = new Map();
  for (const note of notes) {
    const parent = path.posix.dirname(note.path);
    if (!folderBuckets.has(parent)) {
      folderBuckets.set(parent, []);
    }
    folderBuckets.get(parent).push(note);
  }

  for (const [, bucket] of folderBuckets) {
    if (bucket.length < 3) continue;

    const orphanCount = bucket.filter(
      (note) => note.rawLinks.length === 0 && (incomingCounts.get(note.id) ?? 0) === 0,
    ).length;
    if (orphanCount / bucket.length < 0.6) continue;

    const sorted = [...bucket].sort((left, right) => left.path.localeCompare(right.path));
    for (let index = 0; index < sorted.length; index += 1) {
      const source = sorted[index];
      for (let offset = 1; offset <= 2; offset += 1) {
        const target = sorted[(index + offset) % sorted.length];
        if (target.id === source.id) continue;
        const edgeKey = `${source.id}=>${target.id}`;
        if (seenEdges.has(edgeKey)) continue;
        seenEdges.add(edgeKey);
        resolvedEdges.push({ source: source.id, target: target.id, weight: 0.35, kind:
'sibling' });
        source.outgoing.push(target.id);
        incomingCounts.set(target.id, (incomingCounts.get(target.id) ?? 0) + 1);
      }
    }
  }
```

```
  for (const note of notes) {
    note.outgoing = Array.from(new Set(note.outgoing)).sort();
  }

  //  Topology-based community detection (Louvain)
  // Build a graphology undirected graph and run Louvain to detect communities
  // purely from edge structure. This replaces the old folder-name / PM palette
  // approach - communities emerge from what notes *connect to*, not where they sit.
  // Annotate each note with its meaningful "cluster label" (deepest folder segment
  // that makes sense to show in the UI) before passing to detectCommunities.
  for (const note of notes) {
    note._clusterLabel = deriveClusterLabel(note.path);
  }
  const communityMap = detectCommunities(notes, resolvedEdges);
  const topologyCommunityCount = new Set(communityMap.values()).size;

  // Stable color assignment: sort community labels by size descending so the largest
  // communities (most notes) get the most prominent colors in the palette.
  const communitySizes = new Map();
  for (const label of communityMap.values()) {
    communitySizes.set(label, (communitySizes.get(label) ?? 0) + 1);
  }
  const sortedCommunities = Array.from(communitySizes.entries())
    .sort((a, b) => b[1] - a[1])
    .map(([label]) => label);
  const communityColors = new Map(
    sortedCommunities.map((label, index) => [label, DEFAULT_COLORS[index %
DEFAULT_COLORS.length]]),
  );

  //  Obsidian color-group definitions (used alongside Louvain)
  // If the user has tagged notes with Obsidian color groups, those tags take
  // precedence over Louvain for individual notes (intentional manual override).
  const groupDefinitions = buildGroupDefinitions(graphSettings);

  for (const note of notes) {
    note.incomingCount = incomingCounts.get(note.id) ?? 0;
    note.outgoingCount = note.outgoing.length;
    note.degree = note.incomingCount + note.outgoingCount;

    const communityLabel = communityMap.get(note.id) ?? 'Other';
    const communityColor = communityColors.get(communityLabel) ?? DEFAULT_COLORS[0];

    // Obsidian tag-based group overrides topology group when explicitly set
    const matchingGroup = findBestGroup(note.tags, groupDefinitions);
    note.group = matchingGroup?.key ?? communityLabel;
    note.color = matchingGroup?.color ?? communityColor;

    note.importance = Number(
      (
        note.incomingCount * 1.7 +
        note.outgoingCount * 1.15 +
```

```
        note.tags.length * 0.45 +
        Math.min(note.wordCount / 280, 2.5)
      ).toFixed(2)
    );

    delete note.rawLinks;
    delete note.sourceFile;
  }

  notes.sort((left, right) => right.importance - left.importance ||
left.path.localeCompare(right.path));

  const groups = summarizeGroups(notes);
  const payload = {
    generatedAt: new Date().toISOString(),
    vaultRoot,
    noteCount: notes.length,
    edgeCount: resolvedEdges.length,
    groups,
    notes,
    edges: resolvedEdges,
  };

  await fs.writeFile(OUTPUT_PATH, JSON.stringify(payload, null, 2), 'utf8');
  console.log(`Generated ${notes.length} notes and ${resolvedEdges.length} links ->
${OUTPUT_PATH}`);
  console.log(`  Topology communities: ${topologyCommunityCount}; visible groups:
${groups.length}`);
}

//  Louvain community detection

/**
 * Run Louvain community detection and return a Map<noteId, communityLabel>.
 *
 * Strategy:
 *  1. Run Louvain at resolution 0.5 to get an initial partition (often 80-150 communities).
 *  2. Iteratively merge the smallest community into its most-connected neighbor until we
 *     reach TARGET_MAX_COMMUNITIES. Isolated nodes merge into the globally largest
community.
 *  3. Label each surviving community by its dominant _clusterLabel (deepest folder path).
 *     Communities that share a dominant label get a disambiguation suffix only when needed.
 */
function detectCommunities(notes, edges) {
  const TARGET_MAX_COMMUNITIES = 22; // aim for UI-friendly number of color bands
  const LOUVAIN_RESOLUTION = 0.5;   // lower = coarser initial partition

  const G = new Graph({ type: 'undirected', multi: false, allowSelfLoops: false });

  for (const note of notes) {
    G.addNode(note.id, { clusterLabel: note._clusterLabel ?? note.folder });
  }
```

```
  for (const edge of edges) {
    if (!G.hasNode(edge.source) || !G.hasNode(edge.target)) continue;
    if (edge.source === edge.target) continue;
    if (G.hasEdge(edge.source, edge.target)) continue;
    G.addEdge(edge.source, edge.target, { weight: edge.weight ?? 1 });
  }

  if (G.size === 0) {
    return new Map(notes.map((n) => [n.id, 'Vault Root']));
  }

  const rawCommunities = louvain(G, {
    getEdgeWeight: 'weight',
    randomWalk: true,
    resolution: LOUVAIN_RESOLUTION,
  });

  // assignment: Map<nodeId, communityId>  (mutable, numeric IDs throughout)
  const assignment = new Map(Object.entries(rawCommunities));

  // Helper: rebuild a Map<communityId, nodeId[]> from current assignment
  function buildMemberMap() {
    const m = new Map();
    for (const [nodeId, cid] of assignment) {
      if (!m.has(cid)) m.set(cid, []);
      m.get(cid).push(nodeId);
    }
    return m;
  }

  // Iteratively merge the smallest community into its best neighbor
  // until we are at or below TARGET_MAX_COMMUNITIES.
  let memberMap = buildMemberMap();
  while (memberMap.size > TARGET_MAX_COMMUNITIES) {
    // Sort communities ascending by member count
    const sorted = Array.from(memberMap.entries()).sort((a, b) => a[1].length -
b[1].length);
    const [smallestCid, smallestMembers] = sorted[0];

    // Count cross-edges from smallest community to each neighbor community
    const neighborCounts = new Map();
    for (const nodeId of smallestMembers) {
      if (!G.hasNode(nodeId)) continue;
      for (const neighbor of G.neighbors(nodeId)) {
        const neighborCid = assignment.get(neighbor);
        if (neighborCid !== undefined && neighborCid !== smallestCid) {
          neighborCounts.set(neighborCid, (neighborCounts.get(neighborCid) ?? 0) + 1);
        }
      }
    }
```

```
    // Pick target: most cross-edges wins; if isolated, pick the globally largest community
    let targetCid;
    if (neighborCounts.size > 0) {
      targetCid = Array.from(neighborCounts.entries()).sort((a, b) => b[1] - a[1])[0][0];
    } else {
      targetCid = sorted[sorted.length - 1][0]; // largest community
    }

    // Reassign all members of smallest community to target
    for (const nodeId of smallestMembers) {
      assignment.set(nodeId, targetCid);
    }

    // Rebuild only the two affected entries rather than scanning everything
    const targetMembers = memberMap.get(targetCid) ?? [];
    memberMap.delete(smallestCid);
    memberMap.set(targetCid, [...targetMembers, ...smallestMembers]);
  }

  //  Label surviving communities
  // Each community is labeled by the title of its highest-degree node (most
  // connections in the graph). This gives meaningful, readable labels even for
  // orphan-heavy folders where folder paths would all look the same.
  // Ties are broken by note title alphabetically.
  const notesById = new Map(notes.map((n) => [n.id, n]));

  // Sort communities by size desc so largest community gets label priority
  const sortedCids = Array.from(memberMap.keys()).sort((a, b) => {
    return (memberMap.get(b)?.length ?? 0) - (memberMap.get(a)?.length ?? 0);
  });

  const usedLabels = new Set();
  const cidToLabel = new Map();

  for (const cid of sortedCids) {
    const members = memberMap.get(cid);
    if (!members) continue;

    // Find the highest-degree note in this community
    let bestNote = null;
    let bestDegree = -1;
    for (const nodeId of members) {
      const note = notesById.get(nodeId);
      if (!note) continue;
      const deg = G.degree(nodeId);
      if (deg > bestDegree || (deg === bestDegree && (note.title ??
'').localeCompare(bestNote?.title ?? '') < 0)) {
        bestDegree = deg;
        bestNote = note;
      }
    }
```

```
    // Build label: note title, falling back to folder path if note has no title
    const rawTitle = bestNote?.title ?? bestNote?._clusterLabel ?? 'Cluster';
    // Truncate long titles to keep the UI chip readable
    const truncated = rawTitle.length > 40 ? `${rawTitle.slice(0, 38)}...` : rawTitle;

    // Disambiguate if another community already claimed this label
    let label = truncated;
    if (usedLabels.has(label)) {
      let idx = 2;
      while (usedLabels.has(`${truncated} (${idx})`)) idx += 1;
      label = `${truncated} (${idx})`;
    }

    usedLabels.add(label);
    cidToLabel.set(cid, label);
  }

  // Build final Map<noteId, label>
  const result = new Map();
  for (const [nodeId, cid] of assignment) {
    result.set(nodeId, cidToLabel.get(cid) ?? 'Other');
  }

  return result;
}

//  File collection

async function collectMarkdownFiles(rootDirectory, vaultRoot, excludedDirectories = new
Set(), excludedFiles = new Set()) {
  const entries = await fs.readdir(rootDirectory, { withFileTypes: true });
  const files = [];

  for (const entry of entries) {
    const absolutePath = path.join(rootDirectory, entry.name);
    const relativePath = normalizePath(path.relative(vaultRoot, absolutePath));

    if (entry.isDirectory()) {
      if (shouldExcludeDirectory(entry.name, relativePath, excludedDirectories)) {
        continue;
      }

      files.push(...(await collectMarkdownFiles(absolutePath, vaultRoot,
excludedDirectories, excludedFiles)));
      continue;
    }

    if (
      entry.isFile() &&
      entry.name.toLowerCase().endsWith('.md') &&
      !shouldExcludeFile(entry.name, relativePath, excludedFiles)
    ) {
```

```
      files.push(absolutePath);
    }
  }

  return files;
}

async function readGraphSettings(obsidianGraphPath) {
  try {
    const raw = await fs.readFile(obsidianGraphPath, 'utf8');
    return JSON.parse(raw);
  } catch {
    return {};
  }
}

async function readConfig() {
  try {
    const raw = await fs.readFile(CONFIG_PATH, 'utf8');
    return JSON.parse(raw);
  } catch {
    return {};
  }
}

async function resolveVaultRoot(config) {
  const configuredPath = process.env.CEREBRO_VAULT_PATH ?? config.vaultPath;
  if (configuredPath) {
    const resolved = path.resolve(APP_ROOT, configuredPath);
    await assertVaultRoot(resolved);
    return resolved;
  }

  const discovered = await findVaultUpTree(APP_ROOT);
  if (discovered) {
    return discovered;
  }

  throw new Error(
    [
      'Unable to resolve the Obsidian vault path.',
      'Set CEREBRO_VAULT_PATH or create cerebro.config.json with {"vaultPath":
"C:/path/to/vault"}.',
    ].join(' '),
  );
}

async function assertVaultRoot(candidatePath) {
  const obsidianPath = path.join(candidatePath, '.obsidian');

  try {
    const stat = await fs.stat(obsidianPath);
```

```
    if (!stat.isDirectory()) {
      throw new Error();
    }
  } catch {
    throw new Error(`Resolved vault path does not contain an .obsidian directory:
${candidatePath}`);
  }
}

async function findVaultUpTree(startPath) {
  let currentPath = startPath;

  while (true) {
    const obsidianPath = path.join(currentPath, '.obsidian');
    try {
      const stat = await fs.stat(obsidianPath);
      if (stat.isDirectory()) {
        return currentPath;
      }
    } catch {
      // Keep walking upward.
    }

    const parentPath = path.dirname(currentPath);
    if (parentPath === currentPath) {
      return null;
    }

    currentPath = parentPath;
  }
}

function buildExcludedDirectories(config) {
  const configured = Array.isArray(config.excludeDirectories)
    ? config.excludeDirectories.map((value) => String(value))
    : [];

  return new Set([...DEFAULT_EXCLUDED_DIRECTORIES, ...configured]);
}

function buildExcludedFiles(config) {
  const configured = Array.isArray(config.excludeFiles) ? config.excludeFiles.map((value) =>
normalizePath(String(value))) : [];
  return new Set(configured.filter(Boolean));
}

function shouldExcludeDirectory(entryName, relativePath, excludedDirectories) {
  if (excludedDirectories.has(entryName) || excludedDirectories.has(relativePath)) {
    return true;
  }

  return entryName.startsWith('.') && entryName !== '.obsidian';
```

```
}

function shouldExcludeFile(entryName, relativePath, excludedFiles) {
  if (excludedFiles.has(entryName) || excludedFiles.has(relativePath)) {
    return true;
  }

  return false;
}

function buildGroupDefinitions(graphSettings) {
  const colorGroups = Array.isArray(graphSettings?.colorGroups) ? graphSettings.colorGroups
: [];

  return colorGroups
    .map((group) => {
      const match = typeof group?.query === 'string' ? group.query.match(/^tag:#(.+)$/) :
null;
      if (!match) {
        return null;
      }

      return {
        key: match[1],
        color: rgbNumberToHex(group?.color?.rgb),
      };
    })
    .filter(Boolean);
}

function buildLookupMap(notes, picker) {
  const values = new Map();

  for (const note of notes) {
    for (const value of picker(note)) {
      const normalized = normalizePath(String(value).replace(/\.md$/i, '').trim());
      if (!normalized) {
        continue;
      }

      const existing = values.get(normalized);
      if (!existing) {
        values.set(normalized, [note.id]);
      } else {
        existing.push(note.id);
      }
    }
  }

  return values;
}
```

```
function resolveNoteTarget(rawTarget, sourcePath, notesById, basenameMap, aliasMap) {
  const normalized = normalizePath(rawTarget.replace(/\.md$/i, '').trim());
  if (!normalized) {
    return null;
  }

  if (notesById.has(normalized)) {
    return normalized;
  }

  const sourceDirectory = path.posix.dirname(sourcePath);
  const relativeCandidate =
normalizePath(path.posix.normalize(path.posix.join(sourceDirectory, normalized)));
  if (notesById.has(relativeCandidate)) {
    return relativeCandidate;
  }

  const basenameMatches = basenameMap.get(path.posix.basename(normalized));
  if (basenameMatches?.length === 1) {
    return basenameMatches[0];
  }

  const aliasMatches = aliasMap.get(normalized);
  if (aliasMatches?.length === 1) {
    return aliasMatches[0];
  }

  return null;
}

function extractTitle(content, fallback) {
  const match = content.match(/^#\s+(.+)$/m);
  return match?.[1]?.trim() || fallback;
}

function extractLinks(content) {
  return Array.from(content.matchAll(WIKILINK_PATTERN), (match) => match[1].trim());
}

function extractInlineTags(content) {
  const tags = new Set();
  for (const match of content.matchAll(INLINE_TAG_PATTERN)) {
    tags.add(match[2].trim());
  }
  return Array.from(tags).sort();
}

function extractExcerpt(content) {
  const cleanedLines = content
    .split(/\r?\n/)
    .map((line) => line.trim())
    .filter((line) => line && !line.startsWith('#') && !line.startsWith('> ') &&
```

```
!line.startsWith('|') && !line.startsWith('- '));

  const excerpt = cleanedLines
    .join(' ')
    .replace(/\[\[([^\]|#]+)(?:#[^\]|]+)?(?:\|([^\]]+))?\]\]/g, (_, target, label) => label
|| path.posix.basename(target))
    .replace(/\[(.*?)\]\((.*?)\)/g, '$1')
    .replace(/[`*_>]/g, '')
    .replace(/\s+/g, ' ')
    .trim();

  return excerpt.slice(0, 280);
}

function countWords(content) {
  const text = content.replace(/\s+/g, ' ').trim();
  return text ? text.split(' ').length : 0;
}

function toStringList(value) {
  if (Array.isArray(value)) {
    return value.map((entry) => String(entry).trim()).filter(Boolean);
  }

  if (typeof value === 'string') {
    return value
      .split(',')
      .map((entry) => entry.replace(/[[\]]/g, '').trim())
      .filter(Boolean);
  }

  return [];
}

function summarizeGroups(notes) {
  const groups = new Map();

  for (const note of notes) {
    const existing = groups.get(note.group);
    if (existing) {
      existing.count += 1;
      continue;
    }

    groups.set(note.group, {
      key: note.group,
      label: note.group,
      color: note.color,
      count: 1,
    });
  }
```

```
  return Array.from(groups.values()).sort((left, right) => right.count - left.count ||
left.label.localeCompare(right.label));
}

function findBestGroup(tags, groups) {
  return groups
    .filter((group) => tags.some((tag) => tag === group.key ||
tag.startsWith(`${group.key}/`)))
    .sort((left, right) => right.key.length - left.key.length)[0];
}

/**
 * Returns a fallback cluster label for a note path (used only when a note
 * has degree 0 and no title can be derived from the graph).
 * Uses the immediate parent directory name.
 */
function deriveClusterLabel(notePath) {
  const segments = notePath.split('/');
  if (segments.length === 1) return 'Vault Root';
  return segments[segments.length - 2];
}

function normalizePath(value) {
  return value.replace(/\\/g, '/').replace(/^\.?\//, '').trim();
}

function rgbNumberToHex(value) {
  if (typeof value !== 'number' || Number.isNaN(value)) {
    return DEFAULT_COLORS[0];
  }

  return `#${value.toString(16).padStart(6, '0')}`;
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

SHA256 incremental cache

Incremental parsing is handled by `scripts/cache-manager.mjs`. The cache key is the SHA256 of file contents, not the file path, so unchanged notes survive renames and only stale hashes are evicted at the end of a run.

Full `scripts/cache-manager.mjs`

```
/**
 * cache-manager.mjs
 *
 * SHA256-based incremental parse cache for build-vault-graph.mjs.
```

```
 *
 * Each markdown file is hashed (content only). If a matching cache entry
 * exists under .cerebro-cache/<sha256>.json, the cached parsed result is
 * returned directly - skipping the gray-matter + link-extraction work.
 *
 * Cache files are cheap (a few KB each) and safe to delete at any time;
 * the next build will regenerate them. Add .cerebro-cache/ to .gitignore.
 *
 * Usage:
 *   import { getCached, saveCache } from './cache-manager.mjs';
 *
 *   const cached = await getCached(raw);      // null -> not cached
 *   if (cached) { ...use cached... }
 *   else        { ...parse...; await saveCache(raw, parsedResult); }
 */

import crypto from 'node:crypto';
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const APP_ROOT = path.resolve(__dirname, '..');
const CACHE_DIR = path.join(APP_ROOT, '.cerebro-cache');

let cacheReady = false;

/** Lazily create the cache directory. */
async function ensureCacheDir() {
  if (cacheReady) return;
  await fs.mkdir(CACHE_DIR, { recursive: true });
  cacheReady = true;
}

/** SHA256 hex digest of a UTF-8 string. Exported so callers can hash once and reuse. */
export function sha256(content) {
  return crypto.createHash('sha256').update(content, 'utf8').digest('hex');
}

/** Cache file path for a given content hash. */
function cachePath(hash) {
  return path.join(CACHE_DIR, `${hash}.json`);
}

/**
 * Return a previously-cached parse result for `raw`, or null if the file has
 * changed (or was never cached).
 *
 * @param {string} raw - Raw file content
 * @returns {Promise<object | null>}
 */
```

```
export async function getCached(raw) {
  return getCachedByHash(sha256(raw));
}

/**
 * Like getCached, but accepts a pre-computed hash to avoid double-hashing.
 * Use this when you've already called sha256(raw) for tracking purposes.
 *
 * @param {string} hash - SHA256 hex digest
 * @returns {Promise<object | null>}
 */
export async function getCachedByHash(hash) {
  await ensureCacheDir();
  try {
    const json = await fs.readFile(cachePath(hash), 'utf8');
    return JSON.parse(json);
  } catch {
    return null;
  }
}

/**
 * Write a parse result to the cache keyed by `raw`'s SHA256.
 *
 * @param {string} raw - Raw file content (used to derive the cache key)
 * @param {object} result - Parsed result to persist
 */
export async function saveCache(raw, result) {
  await saveCacheByHash(sha256(raw), result);
}

/**
 * Like saveCache, but accepts a pre-computed hash.
 *
 * @param {string} hash - SHA256 hex digest
 * @param {object} result - Parsed result to persist
 */
export async function saveCacheByHash(hash, result) {
  await ensureCacheDir();
  await fs.writeFile(cachePath(hash), JSON.stringify(result), 'utf8');
}

/**
 * Evict all cache entries not in the provided set of current hashes.
 * Call this at the end of a build pass to prune stale entries.
 *
 * @param {Set<string>} activeHashes - Hashes seen during this build run
 */
export async function evictStaleCache(activeHashes) {
  await ensureCacheDir();
  let entries;
  try {
```

```
    entries = await fs.readdir(CACHE_DIR);
  } catch {
    return;
  }

  for (const entry of entries) {
    if (!entry.endsWith('.json')) continue;
    const hash = entry.slice(0, -5); // strip .json
    if (!activeHashes.has(hash)) {
      await fs.unlink(path.join(CACHE_DIR, entry)).catch(() => {});
    }
  }
}
```

Semantic edges with graphify

Wikilinks only capture explicit note-to-note references. Graphify adds a second layer of semantic relationships by scanning a folder, generating an internal graph, and writing `graphify-out/graph.json`. CerebroFinnie then imports those edges and merges them into the main graph before clustering.

For a repo-local bootstrap, this repo also includes `scripts/generate-graphify-json.mjs`. That script derives a compatible `graphify-out/graph.json` from the current Cerebro snapshot using title references plus TF-IDF similarity, which is useful when you want semantic enrichment without depending on an external graphify runtime.

Vendored graphify lives in `tools/graphify/`. It can be installed locally without cloning anything else:

```
cd tools/graphify
pip install -r requirements.txt
```

In this repo revision, the vendored Python package exposes the reusable graphify modules plus the installer/query CLI surface. The end-to-end semantic extraction flow remains skill-driven. Use the repo-local skill trigger in a supported agent environment:

```
/graphify C:/Users/james/Desktop/CerebroAtlas/pm-brain-main
```

That workflow writes `graphify-out/graph.json`. If you already have a compatible `graphify-out/graph.json` from another graphify-capable environment, you can drop it into the vault and rebuild without rerunning the semantic pass.

The repo-local bootstrap path is:

```
npm run generate-semantic-json
```

Then rebuild the viewer graph:

```
node scripts/build-vault-graph.mjs
```

The imported edges are emitted as:

- `kind: 'semantic'`

- `confidence: 'EXTRACTED' | 'INFERRED' | 'AMBIGUOUS'`
- `relation: '<graphify relation label>'`

The current importer weights them as:

- `EXTRACTED` -> `0.8`
- `INFERRED` -> `0.5`
- `AMBIGUOUS` -> `0.3`

Full `scripts/import-graphify-edges.mjs`

```
/**
 * import-graphify-edges.mjs
 *
 * Reads the graphify output (graphify-out/graph.json) from anywhere inside the
 * vault tree and maps graphify's node IDs back to CerebroFinnie note IDs via
 * source_file path matching.
 *
 * graphify produces edges tagged as:
 *   confidence: 'EXTRACTED' | 'INFERRED' | 'AMBIGUOUS'
 *   relation:   'semantically_similar_to' | 'references' | 'uses' | 'calls' | ...
 *
 * We emit these as kind: 'semantic' edges so the viewer (and Louvain) can treat
 * them differently from wikilinks and sibling chains.
 *
 * Usage (automatic - called by build-vault-graph.mjs):
 *   No graphify output? Returns [] silently.
 *   Graphify output present? Returns merged semantic edge list.
 *
* To generate graphify output, run in a skill-aware agent environment:
*   /graphify C:/path/to/vault/pm-brain-main
*   (or place a compatible graphify-out/graph.json in the vault tree)
 */

import fs from 'node:fs/promises';
import path from 'node:path';

/**
 * Scan the vault tree for any graphify-out/graph.json files, then merge
 * the semantic edges from each into CerebroFinnie's edge format.
 *
 * @param {string} vaultRoot - Absolute path to the Obsidian vault root
 * @param {Array<{id: string, path: string}>} notes - CerebroFinnie notes array
 * @param {Set<string>} seenEdges - Already-seen edge keys to prevent duplicates
 * @returns {Promise<Array>} Semantic edges ready to push into resolvedEdges
 */
export async function importGraphifyEdges(vaultRoot, notes, seenEdges) {
  const graphifyFiles = await findGraphifyOutputs(vaultRoot);
  if (graphifyFiles.length === 0) {
    return [];
  }
```

```
  // Build a lookup from source_file (relative to vault root) -> note id
  const pathToNoteId = new Map();
  for (const note of notes) {
    // note.path is already relative to vaultRoot, normalized with /
    pathToNoteId.set(note.path, note.id);
    // Also index by filename stem for basename-only matches
    pathToNoteId.set(path.posix.basename(note.path), note.id);
  }

  const semanticEdges = [];

  for (const graphifyPath of graphifyFiles) {
    let data;
    try {
      const raw = await fs.readFile(graphifyPath, 'utf8');
      data = JSON.parse(raw);
    } catch {
      continue; // skip malformed files
    }

    const edges = data.edges ?? [];
    const nodes = data.nodes ?? [];

    // Build graphify nodeId -> source_file map
    const nodeSourceFile = new Map();
    for (const node of nodes) {
      if (node.id && node.source_file) {
        nodeSourceFile.set(node.id, node.source_file);
      }
    }

    for (const edge of edges) {
      if (!edge.source || !edge.target) continue;

      const srcFile = nodeSourceFile.get(edge.source);
      const tgtFile = nodeSourceFile.get(edge.target);
      if (!srcFile || !tgtFile) continue;

      const srcId = resolveToNoteId(srcFile, vaultRoot, pathToNoteId);
      const tgtId = resolveToNoteId(tgtFile, vaultRoot, pathToNoteId);
      if (!srcId || !tgtId || srcId === tgtId) continue;

      const edgeKey = `${srcId}=>${tgtId}`;
      if (seenEdges.has(edgeKey)) continue;
      seenEdges.add(edgeKey);

      // Weight: EXTRACTED = 0.8, INFERRED = 0.5, AMBIGUOUS = 0.3
      const confidence = edge.confidence ?? 'INFERRED';
      const weight = confidence === 'EXTRACTED' ? 0.8 : confidence === 'INFERRED' ? 0.5 :
0.3;

      semanticEdges.push({
```

```
        source: srcId,
        target: tgtId,
        weight,
        kind: 'semantic',
        confidence,
        relation: edge.relation ?? 'related_to',
      });
    }
  }

  return semanticEdges;
}

/**
 * Recursively find all graphify-out/graph.json files under the vault root.
 */
async function findGraphifyOutputs(dir) {
  const results = [];
  let entries;
  try {
    entries = await fs.readdir(dir, { withFileTypes: true });
  } catch {
    return results;
  }

  for (const entry of entries) {
    const fullPath = path.join(dir, entry.name);
    if (entry.isDirectory()) {
      // Skip hidden dirs and the viewer app itself
      if (entry.name.startsWith('.') || entry.name === 'node_modules') continue;
      results.push(...(await findGraphifyOutputs(fullPath)));
    } else if (entry.isFile() && entry.name === 'graph.json') {
      // Check if parent folder is graphify-out
      if (path.basename(path.dirname(fullPath)) === 'graphify-out') {
        results.push(fullPath);
      }
    }
  }

  return results;
}

/**
 * Map a graphify source_file path -> a CerebroFinnie note ID.
 * Tries: absolute match -> relative-to-vault match -> basename match.
 */
function resolveToNoteId(sourceFile, vaultRoot, pathToNoteId) {
  if (!sourceFile) return null;

  // Normalize separators
  const normalized = sourceFile.replace(/\\/g, '/');
```

```
  // Try direct path match (relative, no extension)
  const withoutExt = normalized.replace(/\.md$/i, '');
  if (pathToNoteId.has(withoutExt)) return pathToNoteId.get(withoutExt);

  // Try stripping vaultRoot prefix
  const vaultNormalized = vaultRoot.replace(/\\/g, '/').replace(/\/?$/, '/');
  if (normalized.startsWith(vaultNormalized)) {
    const relative = normalized.slice(vaultNormalized.length).replace(/\.md$/i, '');
    if (pathToNoteId.has(relative)) return pathToNoteId.get(relative);
  }

  // Try basename match
  const basename = path.posix.basename(normalized, '.md');
  if (pathToNoteId.has(basename)) return pathToNoteId.get(basename);

  return null;
}
```

Louvain clustering

Folder-driven grouping has been replaced with topology-driven clustering using `graphology` and `graphology-communities-louvain`. The builder creates an undirected graph from notes plus all resolved edges, runs Louvain, and then iteratively merges micro-communities until the total falls below a UI-friendly ceiling.

The current implementation uses:

- `LOUVAIN_RESOLUTION = 0.5`
- `TARGET_MAX_COMMUNITIES = 22`

Each community is labeled by the title of its highest-degree note, which produces readable group chips like actual topic names instead of folder labels. Obsidian graph color groups still override color for notes that are deliberately tagged.

Louvain implementation excerpt

```
function detectCommunities(notes, edges) {
  const TARGET_MAX_COMMUNITIES = 22; // aim for UI-friendly number of color bands
  const LOUVAIN_RESOLUTION = 0.5;   // lower = coarser initial partition

  const G = new Graph({ type: 'undirected', multi: false, allowSelfLoops: false });

  for (const note of notes) {
    G.addNode(note.id, { clusterLabel: note._clusterLabel ?? note.folder });
  }

  for (const edge of edges) {
    if (!G.hasNode(edge.source) || !G.hasNode(edge.target)) continue;
    if (edge.source === edge.target) continue;
    if (G.hasEdge(edge.source, edge.target)) continue;
    G.addEdge(edge.source, edge.target, { weight: edge.weight ?? 1 });
  }
```

```
  if (G.size === 0) {
    return new Map(notes.map((n) => [n.id, 'Vault Root']));
  }

  const rawCommunities = louvain(G, {
    getEdgeWeight: 'weight',
    randomWalk: true,
    resolution: LOUVAIN_RESOLUTION,
  });

  // assignment: Map<nodeId, communityId>  (mutable, numeric IDs throughout)
  const assignment = new Map(Object.entries(rawCommunities));

  // Helper: rebuild a Map<communityId, nodeId[]> from current assignment
  function buildMemberMap() {
    const m = new Map();
    for (const [nodeId, cid] of assignment) {
      if (!m.has(cid)) m.set(cid, []);
      m.get(cid).push(nodeId);
    }
    return m;
  }

  // Iteratively merge the smallest community into its best neighbor
  // until we are at or below TARGET_MAX_COMMUNITIES.
  let memberMap = buildMemberMap();
  while (memberMap.size > TARGET_MAX_COMMUNITIES) {
    // Sort communities ascending by member count
    const sorted = Array.from(memberMap.entries()).sort((a, b) => a[1].length -
b[1].length);
    const [smallestCid, smallestMembers] = sorted[0];

    // Count cross-edges from smallest community to each neighbor community
    const neighborCounts = new Map();
    for (const nodeId of smallestMembers) {
      if (!G.hasNode(nodeId)) continue;
      for (const neighbor of G.neighbors(nodeId)) {
        const neighborCid = assignment.get(neighbor);
        if (neighborCid !== undefined && neighborCid !== smallestCid) {
          neighborCounts.set(neighborCid, (neighborCounts.get(neighborCid) ?? 0) + 1);
        }
      }
    }

    // Pick target: most cross-edges wins; if isolated, pick the globally largest community
    let targetCid;
    if (neighborCounts.size > 0) {
      targetCid = Array.from(neighborCounts.entries()).sort((a, b) => b[1] - a[1])[0][0];
    } else {
      targetCid = sorted[sorted.length - 1][0]; // largest community
    }
```

```
    // Reassign all members of smallest community to target
    for (const nodeId of smallestMembers) {
      assignment.set(nodeId, targetCid);
    }

    // Rebuild only the two affected entries rather than scanning everything
    const targetMembers = memberMap.get(targetCid) ?? [];
    memberMap.delete(smallestCid);
    memberMap.set(targetCid, [...targetMembers, ...smallestMembers]);
  }

  //  Label surviving communities
  // Each community is labeled by the title of its highest-degree node (most
  // connections in the graph). This gives meaningful, readable labels even for
  // orphan-heavy folders where folder paths would all look the same.
  // Ties are broken by note title alphabetically.
  const notesById = new Map(notes.map((n) => [n.id, n]));

  // Sort communities by size desc so largest community gets label priority
  const sortedCids = Array.from(memberMap.keys()).sort((a, b) => {
    return (memberMap.get(b)?.length ?? 0) - (memberMap.get(a)?.length ?? 0);
  });

  const usedLabels = new Set();
  const cidToLabel = new Map();

  for (const cid of sortedCids) {
    const members = memberMap.get(cid);
    if (!members) continue;

    // Find the highest-degree note in this community
    let bestNote = null;
    let bestDegree = -1;
    for (const nodeId of members) {
      const note = notesById.get(nodeId);
      if (!note) continue;
      const deg = G.degree(nodeId);
      if (deg > bestDegree || (deg === bestDegree && (note.title ??
'').localeCompare(bestNote?.title ?? '') < 0)) {
        bestDegree = deg;
        bestNote = note;
      }
    }

    // Build label: note title, falling back to folder path if note has no title
    const rawTitle = bestNote?.title ?? bestNote?._clusterLabel ?? 'Cluster';
    // Truncate long titles to keep the UI chip readable
    const truncated = rawTitle.length > 40 ? `${rawTitle.slice(0, 38)}...` : rawTitle;

    // Disambiguate if another community already claimed this label
    let label = truncated;
```

```
    if (usedLabels.has(label)) {
      let idx = 2;
      while (usedLabels.has(`${truncated} (${idx})`)) idx += 1;
      label = `${truncated} (${idx})`;
    }

    usedLabels.add(label);
    cidToLabel.set(cid, label);
  }

  // Build final Map<noteId, label>
  const result = new Map();
  for (const [nodeId, cid] of assignment) {
    result.set(nodeId, cidToLabel.get(cid) ?? 'Other');
  }

  return result;
}

//  File collection
```

The 3D viewer

The viewer is a React 19 + Vite + TypeScript app using Three.js through `@react-three/fiber` and `@react-three/drei`.

Key patterns in the UI layer:

- Three topology modes: `centralized`, `clustered`, and `distributed`.
- Search and group filtering in `ControlHud`.
- Right-panel note inspection in `NoteInspector`.
- Full-note modal rendering in `NoteModal`.
- Optional hand gesture navigation through MediaPipe in `useHandNavigation`.
- Path-based docs routing in `src/main.tsx`: `/docs` renders a public methodology page with no authentication requirement.

The graph data contract now includes edge provenance:

```
export interface VaultEdge {
  source: string;
  target: string;
  weight: number;
  kind?: 'wikilink' | 'semantic' | 'sibling';
  confidence?: 'EXTRACTED' | 'INFERRED' | 'AMBIGUOUS';
  relation?: string;
}
```

Private sync with Supabase

The raw vault stays local. Only the generated `vault-graph.json` snapshot is published. The deployed app can either read local data in development or authenticate against Supabase and fetch the latest private snapshot through

`api/snapshot.js` .

Environment variables accepted by the current implementation:

- Client runtime: `VITE_SUPABASE_URL` plus either `VITE_SUPABASE_PUBLISHABLE_KEY` or `VITE_SUPABASE_ANON_KEY`
- Server runtime: `SUPABASE_URL` plus either `SUPABASE_SECRET_KEY` or `SUPABASE_SERVICE_ROLE_KEY`
- Optional: `SUPABASE_SNAPSHOT_BUCKET` , `SUPABASE_SNAPSHOT_PATH` , `CEREBRO_ALLOWED_EMAIL`

One-time publish command:

```
node --env-file=.env scripts/publish-snapshot.mjs
```

Example `.env` :

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_SNAPSHOT_BUCKET=cerebro-private
SUPABASE_SNAPSHOT_PATH=snapshots/latest/vault-graph.json
```

Full `scripts/publish-snapshot.mjs`

```
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { spawn } from 'node:child_process';
import { createClient } from '@supabase/supabase-js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const APP_ROOT = path.resolve(__dirname, '..');
const SNAPSHOT_PATH = path.join(APP_ROOT, 'public', 'data', 'vault-graph.json');
const DEFAULT_BUCKET = 'cerebro-private';
const DEFAULT_REMOTE_PATH = 'snapshots/latest/vault-graph.json';

async function main() {
  const supabaseUrl = readRequiredEnv('SUPABASE_URL');
  const supabaseSecretKey = readRequiredEnv('SUPABASE_SECRET_KEY',
'SUPABASE_SERVICE_ROLE_KEY');
  const snapshotBucket = process.env.SUPABASE_SNAPSHOT_BUCKET?.trim() || DEFAULT_BUCKET;
  const remotePath = process.env.SUPABASE_SNAPSHOT_PATH?.trim() || DEFAULT_REMOTE_PATH;

  await runNodeScript(path.join(APP_ROOT, 'scripts', 'build-vault-graph.mjs'));

  const snapshotText = await fs.readFile(SNAPSHOT_PATH, 'utf8');
  const supabase = createClient(supabaseUrl, supabaseSecretKey, {
    auth: {
      autoRefreshToken: false,
      persistSession: false,
    },
  });
```

```
  await ensureBucket(supabase, snapshotBucket);

  const snapshotBlob = new Blob([snapshotText], { type: 'application/json' });
  const { error: uploadError } = await
supabase.storage.from(snapshotBucket).upload(remotePath, snapshotBlob, {
    cacheControl: '60',
    contentType: 'application/json',
    upsert: true,
  });

  if (uploadError) {
    throw uploadError;
  }

  console.log(`Uploaded snapshot to supabase://${snapshotBucket}/${remotePath}`);
}

async function ensureBucket(supabase, bucketName) {
  const { data: buckets, error: bucketsError } = await supabase.storage.listBuckets();
  if (bucketsError) {
    throw bucketsError;
  }

  if (buckets?.some((bucket) => bucket.name === bucketName)) {
    return;
  }

  const { error: createError } = await supabase.storage.createBucket(bucketName, { public:
false });
  if (createError && createError.message !== 'The resource already exists') {
    throw createError;
  }
}

function readRequiredEnv(...names) {
  for (const name of names) {
    const value = process.env[name]?.trim();
    if (value) {
      return value;
    }
  }

  throw new Error(`Missing required environment variable: ${names.join(' or ')}`);
}

function runNodeScript(scriptPath) {
  return new Promise((resolve, reject) => {
    const child = spawn(process.execPath, [scriptPath], {
      cwd: APP_ROOT,
      stdio: 'inherit',
      env: process.env,
```

```
    });

    child.on('error', reject);
    child.on('close', (code) => {
      if (code === 0) {
        resolve(undefined);
        return;
      }

      reject(new Error(`Script exited with code ${code ?? 'unknown'}`));
    });
  });
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

Bank and enterprise deployment model

This pattern maps unusually well to regulated enterprises.

- Private connectivity. Run the stack behind Azure Private Link or equivalent private networking.
- Identity. Use Entra ID OIDC for user auth and role-bound access to published knowledge surfaces.
- Models. Use Azure OpenAI, Anthropic via Bedrock, or an internal model endpoint for semantic extraction.
- Auditability. Because the wiki is Markdown plus git history, every knowledge mutation is diffable and attributable.
- Governance. This directly supports SR 11-7 style lineage expectations and EU AI Act audit-trace requirements.
- Domain spines. Stand up separate vaults per business line such as Markets, Wealth, Risk, Commercial Credit, and Compliance while keeping the same build and viewer pattern.

In a bank, the graph becomes more than a personal knowledge map. It becomes a navigable index over policies, controls, incidents, counterparties, initiatives, and regulatory obligations.

End-to-end walkthrough

1. Install the app repo

```
gh repo clone JPFinnie/CerebroFinnie
cd CerebroFinnie
npm install
```

2. Point the app at the vault

```
copy cerebro.config.example.json cerebro.config.json
```

Set `vaultPath` to the absolute or relative path of your Obsidian vault.

3. Build the local graph snapshot

```
node scripts/build-vault-graph.mjs
```

Expected behavior:

- first run parses everything and fills `.cerebro-cache/`
- later runs should show high cache-hit counts
- output lands at `public/data/vault-graph.json`

4. Run locally

```
npm run dev
```

Open `http://localhost:5173`. The main app runs at `/`. The methodology page runs at `/docs` and does not require authentication.

5. Add semantic edges

```
cd tools/graphify
pip install -r requirements.txt
cd ../..
node scripts/build-vault-graph.mjs
```

Run `/graphify C:/Users/james/Desktop/CerebroAtlas/pm-brain-main` in a skill-aware agent environment before the rebuild. After a successful semantic pass, the rebuild should report merged semantic edges and may produce stronger Louvain communities.

6. Build for production

```
npm run build
```

The current Vercel config uses a filesystem-first SPA fallback so `/docs` works without breaking `/api/*` or static files.

7. Publish the private snapshot

```
node --env-file=.env scripts/publish-snapshot.mjs
```

8. Deploy the site

Deploy the Vite app to Vercel. The docs page is public, while the main graph can stay auth-gated through Supabase if runtime credentials are configured.

9. Export the methodology as PDF

```
npx md-to-pdf docs/METHODOLOGY.md
```

Verification checklist

- `npm install`
- `node scripts/build-vault-graph.mjs`
- run the build a second time and confirm cache hits dominate
- `cd tools/graphify && pip install -r requirements.txt`
- run `/graphify C:/Users/james/Desktop/CerebroAtlas/pm-brain-main` in a skill-aware agent environment
- rebuild and confirm semantic edges merge cleanly
- `npm run build`
- `npx md-to-pdf docs/METHODOLOGY.md`
- `node --env-file=.env scripts/publish-snapshot.mjs`

Delivery modes for colleagues without GitHub access

- Public URL: https://cerebro-finnie.vercel.app/docs
- Standalone Markdown: email `docs/METHODOLOGY.md`
- PDF export: `npx md-to-pdf docs/METHODOLOGY.md`

The document's job is to communicate the pattern and the exact operational steps. Once those are stable, the implementation can evolve without losing the method.